

Class 6: R Functions

Derek Zhang (PID: A17819201)

Table of contents

Q1. Write your first R function: <code>add()</code>	1
Q2. Write a <code>generate_dna()</code> function	2
Q3. Write a <code>generate_protein()</code> function	4
Q4. <code>generate_protein()</code> for lengths 6 to 13	5
Q5. BLASTp search for generated proteins	5
Q6. Connecting findings to immunology	7

Q1. Write your first R function: `add()`

Here is the `add()` function, with default fall-back values that will be used when no input value is specified for the argument.

```
add1 <- function(int1 = 0, int2 = 0){  
  return(int1 + int2) #directly returns the summed values in addition arithmetic  
}  
add2 <- function(vector1 = c()){  
  return(sum(vector1)) #because vector1 argument takes c(x, y, z, ...) as an argument,  
  #it will pass in a vector that can be summed directly  
}  
add3 <- function(...){ #the ellipses allow storage of unspecified  
  #number of variables as a list-like object  
  return(sum(...)) # the ellipses provide the multiple variables in  
  #a form that is treated as a list by other functions, so sum(...)  
  #will add the elements together, treating it as a list  
}  
  
#The functions are ran here  
add1(1, 2) #should return 3
```

```
[1] 3
```

```
add2(c(1, 2, 3)) #should return 6
```

```
[1] 6
```

```
add3(1, 2, 3, 4, 5, -15) #should return 0
```

```
[1] 0
```

Q2. Write a generate_dna() function

```
generate_dna1 = function(length_int = 0){
  DNA <- c("A", "T", "C", "G") #serves as a "bank" for nucleotides
  newlist <- c() # the base of the new list
  for (i in 1:length_int){ #allows flexible iterations at length_int
    #times
    #the for loop bypasses the need to set replace=TRUE, to be
    #honest I wrote this function before we got to the
    #replace_argument in the class
    newlist <- c(newlist, sample(DNA, 1)) # concatenates the list
    #sequentially, sample(vec, #) serves to randomly sample one
    #element from the vector, which allows us to access the DNA
    #vector as a bank and take elements from it randomly, and
    #concatenate to the new list
  }
  return(newlist)
}

generate_dna2 = function(length_int = 0, string = FALSE){
  DNA <- c("A", "T", "C", "G") #serves as a "bank" for nucleotides
  newlist <- c()
  for (i in 1:length_int){ #allows flexible iterations at length_int
    #times
    newlist <- c(newlist, sample(DNA, 1)) # concatenates the list
    #sequentially, sample(vec, #) serves to randomly sample one
    #element from the vector, which allows us to access the DNA
    #vector as a bank and take elements from it randomly, and
    #concatenate to the new list
  }
}
```

```

}
if(string){
  newlist <- paste(newlist, collapse = "")
}
return(newlist)
}

generate_dna3 = function(length_int = 0, string = FALSE){ # sets a
  #default value for the boolean of string, which is what makes
  #returning a string optional here
  DNA <- c("A", "T", "C", "G") #serves as a "bank" for nucleotides
  newlist <- c()
  for (i in 1:length_int){ #allows flexible iterations at length_int
    #times
    newlist <- c(newlist, sample(DNA, 1)) # concatenates the list
    #sequentially, sample(vec, #) serves to randomly sample one
    #element from the vector, which allows us to access the DNA
    #vector as a bank and take elements from it randomly, and
    #concatenate to the new list
  }
  cat(sprintf(">len%i\n", length_int)) #prints out the >len and
  #formats the integer argument into the string for printing
  cat(paste(newlist, collapse = ""),"\n") #this just prints the
  #string form, concatenated, which is explained below, this is done
  #before just to eliminate the need of another if-statement, its
  #just logical i think, also turns off quotes so the answer can be
  #directly copied
  if(string){ #uses the boolean argument of string to determine if a
    #string should be outputted or not, if yes then the following is ran
    newlist <- paste(newlist, collapse = "") # the paste function
    #joins characters of a vector into a single string, and collapse
    #is what goes in between each element in the string once
    #concatenated, in this case collapse is basically nothing, so
    #that there are no gaps in between elements
  }

  #return(newlist) #just returns the new list, may not be necessary
  #so it is commented out, because the function doesn't ask for it
}

#expected output

```

```
generate_dna1(10) # expects a list of characters including the
```

```
[1] "C" "A" "G" "C" "G" "A" "T" "G" "A" "A"
```

```
#nucleotides of the right length: 10, ex: "A" "T" "A" "T" "A" "T"  
#"A" "T" "A" "T"  
generate_dna2(10, TRUE) # expects a string of characters including
```

```
[1] "CAACAGACGT"
```

```
#the nucleotides of the right length: 10 ex: "ATATATATAT"  
generate_dna3(10, TRUE) # expects a string of characters including
```

```
>len10  
CGTTGTAACC
```

```
#the nucleotides of the right length: 10, as well as a fasta format  
#output of >len10, and then the 10 character dna sequence,  
#ex: >len10  
#. ATATATATAT
```

Q3. Write a generate_protein() function

```
generate_protein <- function(length = 0){ #takes a length argument  
  protein <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I",  
              "L", "K", "M", "F", "P", "S",  
              "T", "W", "Y", "V") #the amino acid bank  
  return (paste(sample(protein, size=length, replace=TRUE),  
                collapse="")) #just uses the sample function to  
  #generate a vector of amino  
  #acids of length: length argument from the protein vector source,  
  #and then uses the paste function to concatenate the vector into a  
  #string with no interruptions: collapse = ""  
}
```

```
generate_protein(10) #should generate a string of length 10, of
```

```
[1] "RPHIHKLICF"
```

```
#amino acids, ex: "EQQEQQEQQ"
```

Q4. generate_protein() for lengths 6 to 13

```
for (i in 6:13){ #iterates for the lengths that we have to print out
  #, in for loop form, keeping things easy for us
  cat(sprintf(">id.%i\n", i)) #formats a print for the specific
  #length of the protein sequence, this is in fasta format, also
  #turns off quotes so the answer can be directly copied
  cat(generate_protein(i),"\n") #generates a protein of the specific
  #length of this iteration, which is i, also turns off quotes so
  #the answer can be directly copied
}
```

```
>id.6
GFGAWD
>id.7
FIFGETW
>id.8
PAVVNYKD
>id.9
SFWDTQAYT
>id.10
WVICMIEKIG
>id.11
ITMDNWQDNFR
>id.12
NPFIKYAGMSFK
>id.13
TFVCSYRQCEFVK
```

Q5. BLASTp search for generated proteins

```
lengths <- c(6:13)
BestIdentities <- c("100%", "100%", "100%", "88.89%", "88.89%",
                  "88.89%", "81.82%", "90.91%")
BestCoverages <- c("100%", "100%", "100%", "100%", "90%",
                  "82%", "92%", "85%")
```

```

Unique <- c("N", "N", "N", "Y", "Y", "Y", "Y", "Y")

##the vectors above are just stored data from the blastp search,
#which are going to be used in the dataframe below
generated <- data.frame(Length = lengths,
                        identity = BestIdentities,
                        coverage <- BestCoverages,
                        unique = Unique) #generates a dataframe

#for the markdown table

generated <- setNames(generated, c("Length (aa)",
                                   "Best hit % identity",
                                   "Best hit % coverage",
                                   "Unique? (Y,N)")) #sets proper
#column names for the dataframe we made so it adheres to the request

knitr::kable(generated) #this uses the knitr library's kable

```

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y,N)
6	100%	100%	N
7	100%	100%	N
8	100%	100%	N
9	88.89%	100%	Y
10	88.89%	90%	Y
11	88.89%	82%	Y
12	81.82%	92%	Y
13	90.91%	85%	Y

```

#function to generate a markdown table from the dataframe we made

```

Q5a. At sequence length of 9 and above, we start to see the best hit identity percentage drop below 100%, **making the hits unique** as at this point in sequence length, it is not the case that both identity and coverage percentage are simultaneously at 100%, and at sequence length of 10 and above we see both the best hit coverage percentage and the best hit identity percentage both be below 100%, which makes them look unique in nature.

Q5b. Very short random peptides of length 6 or a little more are almost always found in the nr database because the sequence space (20^L) is small enough, for example, $20^6 = 64000000$ possible peptides of length 6, that the billions of amino

acids in known proteins have sampled nearly all of them. As peptide length increases, the sequence space grows exponentially which exceeds the size of the known protein universe (approx 10^8 proteins), making it increasingly unlikely that any randomly generated sequence of larger lengths have been observed in nature.

Q6. Connecting findings to immunology

Based on my Q5 results, randomly generated peptides begin to appear unique in nature at around 9-13 amino acids in length, suggesting that MHC class II molecules likely bind peptides of at least this minimum length. Presenting very short peptides (fewer than 9 amino acids) would be a bad choice for the immune system because, as shown in Q5b, short peptides exist in such a small sequence space (20^L) that nearly all possible sequences are already represented somewhere in the known protein universe, which includes the body's own proteins. If the immune system presented and responded to very short peptides, T cells would be constantly activated by fragments that match self-proteins, leading to autoimmune disease. By requiring a minimum peptide length that falls in the unique range, the immune system ensures that only peptides long enough to be genuinely foreign can trigger a T-cell response, thereby distinguishing self from non-self antigens effectively.